Deep Generative Models

Lecture 12

Roman Isachenko

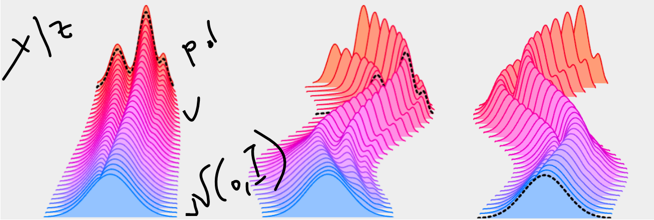
AI Masters

2026, Spring

Recap of Previous Lecture

$\frac{dx}{dt} = v(x, t)$

$$\mathbb{E}_{t \sim U[0,1]} \mathbb{E}_{\mathbf{x} \sim p_t(\mathbf{x})} \underbrace{\left\| \mathbf{v}(\mathbf{x}, t) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t) \right\|^2}_{} \to \min_{\boldsymbol{\theta}}$$



$x|z$

p_1

$\mathcal{N}(0, I)$

Problem: The true vector field $\mathbf{v}(\mathbf{x}, t)$ is **unknown**.

$$p_t(\mathbf{x}) = \int p_t(\mathbf{x}|\mathbf{z}) p(\mathbf{z}) d\mathbf{z}$$

C.E.

$$\frac{\partial p_t(\mathbf{x}|\mathbf{z})}{\partial t} = -\operatorname{div}\left(\mathbf{v}(\mathbf{x}, \mathbf{z}, t) p_t(\mathbf{x}|\mathbf{z})\right).$$

Tong A., et al. Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport, 2023

Recap of Previous Lecture

θ^*

Conditional Flow Matching (CFM)

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{z})}\left\|\mathbf{v}(\mathbf{x},\cancel{\mathbf{z}},t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x},t)\right\|^2 \to \min_{\boldsymbol{\theta}}$$

If $\mathrm{supp}(p_t(\mathbf{x})) = \mathbb{R}^m$, then the optimal value of the FM objective equals the optimum for CFM.

Training

1. Sample a time $t \sim U[0,1]$, $\mathbf{z} \sim p(\mathbf{z})$, $\mathbf{x}_t \sim p_t(\mathbf{x}|\mathbf{z})$.
2. Compute the loss $\mathcal{L} = \left\|\mathbf{v}(\mathbf{x}_t,\mathbf{z},t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t,t)\right\|^2$.

Sampling

1. Sample $\mathbf{x}_0 \sim \mathcal{N}(0,\mathbf{I})$.
2. Solve the ODE to obtain $\mathbf{x}_1$:

$$\mathbf{x}_1 = \psi_1(\mathbf{x}_0) = \texttt{ODESolve}_{v}(\mathbf{x}_0, \boldsymbol{\theta}, t_0 = 0, t_1 = 1).$$

Tong A., et al. Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport, 2023

Recap of Previous Lectures

Theorem

The following vector field generates the probability path $p_t(\mathbf{x})$:

$$\mathbf{v}(\mathbf{x}, t) = \mathbb{E}_{p_t(\mathbf{z}|\mathbf{x})}\mathbf{v}(\mathbf{x}, \mathbf{z}, t) = \int \mathbf{v}(\mathbf{x}, \mathbf{z}, t)\frac{p_t(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p_t(\mathbf{x})}d\mathbf{z}$$

Flow Matching (FM)

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x})}\left\|\mathbf{v}(\mathbf{x}, t) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t)\right\|^2 \to \min_{\boldsymbol{\theta}}$$

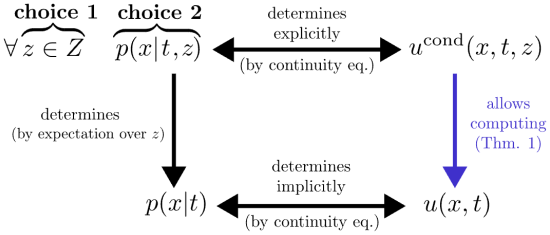
Conditional Flow Matching (CFM)

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{z})}\left\|\mathbf{v}(\mathbf{x}, \mathbf{z}, t) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t)\right\|^2 \to \min_{\boldsymbol{\theta}}$$

Theorem

If $\text{supp}(p_t(\mathbf{x})) = \mathbb{R}^m$, then the optimal value of the FM objective equals the optimum for CFM.

Recap of Previous Lecture



Constraints

$$\mathbb{E}_{p(\mathbf{z})} p_0(\mathbf{x}|\mathbf{z}) = \mathcal{N}(0, \mathbf{I}); \quad \mathbb{E}_{p(\mathbf{z})} p_1(\mathbf{x}|\mathbf{z}) = p_{\text{data}}(\mathbf{x}).$$

- Q1 How should we choose the convenient conditioning latent variable $\mathbf{z}$?
- Q2 How can we parametrize $p_t(\mathbf{x}|\mathbf{z})$ to enforce the constraints?

Gaussian Probability Path and Flow

$$p_t(\mathbf{x}|\mathbf{z}) = \mathcal{N}\left(\boldsymbol{\mu}_t(\mathbf{z}), \boldsymbol{\sigma}_t^2(\mathbf{z})\right) \quad \mathbf{x}_t = \psi_t(\mathbf{x}_0, \mathbf{z}) = \boldsymbol{\mu}_t(\mathbf{z}) + \boldsymbol{\sigma}_t(\mathbf{z}) \odot \boldsymbol{\epsilon}$$

image credit: A Visual Dive into Conditional Flow Matching

Recap of Previous Lecture

Gaussian Conditional Probability Path

$$\underbrace{p_t(\mathbf{x}|\mathbf{z}) = \mathcal{N}\left(\boldsymbol{\mu}_t(\mathbf{z}), \boldsymbol{\sigma}_t^2(\mathbf{z})\right);} \quad \mathbf{x}_t = \boldsymbol{\psi}_t(\mathbf{x}_0, \mathbf{z}) = \boldsymbol{\mu}_t(\mathbf{z}) + \boldsymbol{\sigma}_t(\mathbf{z}) \odot \boldsymbol{\epsilon}$$

$$\boxed{\mathbf{v}(\mathbf{x}_t, \mathbf{z}, t)} = \boldsymbol{\psi}_t'\left(\boldsymbol{\psi}_t(\mathbf{x}_t, \mathbf{z})^{-1}, \mathbf{z}\right) = \boldsymbol{\mu}_t'(\mathbf{z}) + \frac{\boldsymbol{\sigma}_t'(\mathbf{z})}{\boldsymbol{\sigma}_t(\mathbf{z})} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{z}))$$

p_{data}

Conditioning Latent Variable

Let's choose $\mathbf{z} = \mathbf{x}_1$. Then $p(\mathbf{z}) = p_1(\mathbf{x}_1)$.

$$p_t(\mathbf{x}) = \int p_t(\mathbf{x}|\mathbf{x}_1) p_1(\mathbf{x}_1) d\mathbf{x}_1$$

We must ensure the boundary constraints:

$t=0$, $t=1$

$$\begin{cases} \mathbb{E}_{p(\mathbf{z})} p_0(\mathbf{x}|\mathbf{z}) = \mathcal{N}(0, \mathbf{I}) \\ \mathbb{E}_{p(\mathbf{z})} p_1(\mathbf{x}|\mathbf{z}) = p_{\text{data}}(\mathbf{x}) \end{cases} \Rightarrow \begin{cases} p_0(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}(0, \mathbf{I}); \\ p_1(\mathbf{x}|\mathbf{x}_1) = \underbrace{\delta(\mathbf{x} - \mathbf{x}_1).} \end{cases}$$

Lipman Y., et al. Flow Matching for Generative Modeling, 2022

Recap of Previous Lecture

$$p_0(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}(0, \mathbf{I}); \quad p_1(\mathbf{x}|\mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1).$$

Gaussian Conditional Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\boldsymbol{\mu}_t(\mathbf{x}_1), \boldsymbol{\sigma}_t^2(\mathbf{x}_1)\right); \quad \mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_1) + \boldsymbol{\sigma}_t(\mathbf{x}_1) \odot \mathbf{x}_0.$$

Let's consider straight conditional paths:

$$\begin{cases} \boldsymbol{\mu}_t(\mathbf{x}_1) = \underline{t\mathbf{x}_1}; \\ \boldsymbol{\sigma}_t(\mathbf{x}_1) = 1 - t. \end{cases} \quad \Rightarrow \quad \begin{cases} p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1, (1-t)^2\mathbf{I}\right); \\ \mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0. \end{cases}$$

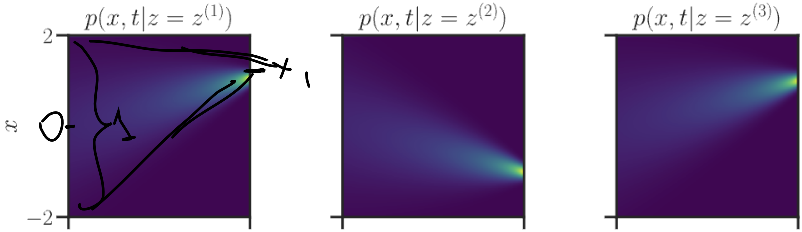


image credit: A Visual Dive into Conditional Flow Matching

Recap of Previous Lecture

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1, (1-t)^2\mathbf{I}\right); \quad \mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$$

$$\boxed{\mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \frac{\mathbf{x}_1 - \mathbf{x}_t}{1-t} = \mathbf{x}_1 - \mathbf{x}_0}$$

$$\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{z})}\left\|\mathbf{v}(\mathbf{x},\mathbf{z},t) - \mathbf{v}_\theta(\mathbf{x},t)\right\|^2 =$$

$$= \mathbb{E}_{\mathbf{x}_1\sim p_{\text{data}}(\mathbf{x})}\mathbb{E}_{\mathbf{x}_0\sim\mathcal{N}(0,\mathbf{I})}\left\|(\mathbf{x}_1 - \mathbf{x}_0) - \mathbf{v}_\theta(t\mathbf{x}_1 + (1-t)\mathbf{x}_0, t)\right\|^2$$

- $\mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t)$ defines straight lines between $p_{\text{data}}(\mathbf{x})$ and $\mathcal{N}(0, \mathbf{I})$.
- The **marginal** path $p_t(\mathbf{x})$ does not give straight lines.

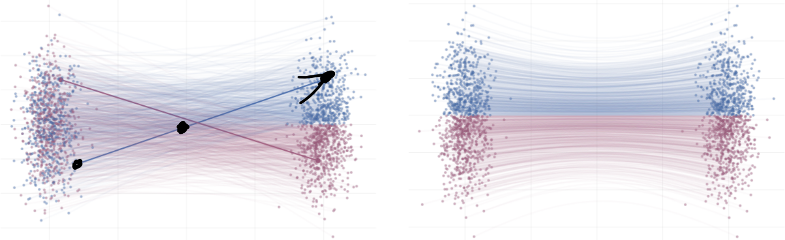


image credit: https://mlg.eng.cam.ac.uk/blog/2024/01/20/flow-matching.html

Recap of Previous Lecture

$$\mathbb{E}_{t \sim U[0,1]} \mathbb{E}_{\mathbf{x}_1 \sim p_{\text{data}}(\mathbf{x})} \mathbb{E}_{\mathbf{x}_0 \sim \mathcal{N}(0, \mathbf{I})} \left\| (\mathbf{x}_1 - \mathbf{x}_0) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) \right\|^2 \to \min_{\boldsymbol{\theta}}$$

Training

1. Sample $\mathbf{x}_1 \sim p_{\text{data}}(\mathbf{x})$.
2. Sample time $t \sim U[0,1]$ and $\mathbf{x}_0 \sim \mathcal{N}(0, \mathbf{I})$.
3. Obtain the noisy image $\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$.
4. Compute the loss $\boxed{\mathcal{L} = \|(\mathbf{x}_1 - \mathbf{x}_0) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)\|^2.}$

Sampling

$x^{l} = x^{l-1} + h \cdot v_\theta(x^{l-1})$

1. Sample $\mathbf{x}_0 \sim \mathcal{N}(0, \mathbf{I})$.
2. Solve the ODE to obtain $\mathbf{x}_1$:

$$\mathbf{x}_1 = \texttt{ODESolve}_v(\mathbf{x}_0, \boldsymbol{\theta}, t_0 = 0, t_1 = 1)$$

Lipman Y., et al. Flow Matching for Generative Modeling, 2022

Outline

$z = x_1$

1. Two-Sided Conditioning
2. Link between Flow Matching and Score-Based Models
3. Discrete Diffusion Models
4. Forward Discrete Process

Diff-text

Outline

Pair Conditioning

$z = x_1$

Conditional Flow Matching

$$\mathbb{E}_{t \sim U[0,1]} \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} \mathbb{E}_{\mathbf{x} \sim p_t(\mathbf{x}|\mathbf{z})} \left\| \mathbf{v}(\mathbf{x}, \mathbf{z}, t) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t) \right\|^2 \to \min_{\boldsymbol{\theta}}$$

Conditioning Latent Variable [Q1]

Let us choose $\boxed{\mathbf{z} = (\mathbf{x}_0, \mathbf{x}_1)}$. Then $p(\mathbf{z}) = p(\mathbf{x}_0, \mathbf{x}_1) = \underbrace{p_0(\mathbf{x}_0) p_1(\mathbf{x}_1)}$.

$$p_t(\mathbf{x}) = \int \underbrace{p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1)}_{p_t(x|z)} \underbrace{p_0(\mathbf{x}_0) p_1(\mathbf{x}_1)}_{p(z)} \underbrace{d\mathbf{x}_0 d\mathbf{x}_1}_{dz}$$

Lipman Y., et al. *Flow Matching for Generative Modeling*, 2022

Pair Conditioning

Conditional Flow Matching

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{z})}\left\|\mathbf{v}(\mathbf{x},\mathbf{z},t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x},t)\right\|^2 \to \min_{\boldsymbol{\theta}}$$

Conditioning Latent Variable [Q1]

Let us choose $\mathbf{z} = (\mathbf{x}_0, \mathbf{x}_1)$. Then $p(\mathbf{z}) = p(\mathbf{x}_0, \mathbf{x}_1) = p_0(\mathbf{x}_0)p_1(\mathbf{x}_1)$.

$$\underset{t=0}{p_t(\mathbf{x})} = \underbrace{\int p_t(\mathbf{x}|\mathbf{x}_0,\mathbf{x}_1)p_0(\mathbf{x}_0)p_1(\mathbf{x}_1)d\mathbf{x}_0 d\mathbf{x}_1}_{p_0(x)} = p_0(x)$$

x_0 x_1

We must enforce boundary constraints:

$$\begin{cases} p_0(\mathbf{x}) = \mathbb{E}_{p(\mathbf{z})}p_0(\mathbf{x}|\mathbf{z}); \\ p_1(\mathbf{x}) = \mathbb{E}_{p(\mathbf{z})}p_1(\mathbf{x}|\mathbf{z}) \end{cases}$$

$p_t(x|x_0, x_1)$

$\boxed{\delta(x - x_0)}$

$\delta(x - x_1)$

Lipman Y., et al. Flow Matching for Generative Modeling, 2022

Pair Conditioning

Conditional Flow Matching

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{z})}\left\|\mathbf{v}(\mathbf{x},\mathbf{z},t)-\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x},t)\right\|^2 \to \min_{\boldsymbol{\theta}}$$

Conditioning Latent Variable [Q1]

Let us choose $\mathbf{z} = (\mathbf{x}_0, \mathbf{x}_1)$. Then $p(\mathbf{z}) = p(\mathbf{x}_0, \mathbf{x}_1) = p_0(\mathbf{x}_0)p_1(\mathbf{x}_1)$.

$$p_t(\mathbf{x}) = \int p_t(\mathbf{x}|\mathbf{x}_0,\mathbf{x}_1)p_0(\mathbf{x}_0)p_1(\mathbf{x}_1)d\mathbf{x}_0 d\mathbf{x}_1$$

We must enforce boundary constraints:

$$\begin{cases} p_0(\mathbf{x}) = \mathbb{E}_{p(\mathbf{z})}p_0(\mathbf{x}|\mathbf{z}); \\ p_1(\mathbf{x}) = \mathbb{E}_{p(\mathbf{z})}p_1(\mathbf{x}|\mathbf{z}) \end{cases} \Rightarrow \boxed{\begin{cases} p_0(\mathbf{x}|\mathbf{x}_0,\mathbf{x}_1) = \delta(\mathbf{x}-\mathbf{x}_0) \\ p_1(\mathbf{x}|\mathbf{x}_0,\mathbf{x}_1) = \delta(\mathbf{x}-\mathbf{x}_1) \end{cases}}$$

Lipman Y., et al. Flow Matching for Generative Modeling, 2022

Two-Sided Conditioning

$$\boxed{p_0(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_0); \quad p_1(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1)}$$

Gaussian Conditional Probability Path

$$\boxed{p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(\underbrace{\boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1)}, \underbrace{\boldsymbol{\sigma}_t^2(\mathbf{x}_0, \mathbf{x}_1)}\right)}; \quad \mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1) + \boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1) \odot \boldsymbol{\epsilon}$$

$$\begin{cases} \mu = x_0(1-t) + t \cdot x_1 \\ \sigma = \varepsilon \cdot \text{[illegible]} \end{cases}$$

image credit: *A Visual Dive into Conditional Flow Matching*

Two-Sided Conditioning

$$p_0(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_0); \quad p_1(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1)$$

Gaussian Conditional Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(\boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1), \boldsymbol{\sigma}_t^2(\mathbf{x}_0, \mathbf{x}_1)\right); \quad \mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1) + \boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1) \odot \boldsymbol{\epsilon}$$

Let's consider straight conditional paths:

$$\begin{cases} \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1) = t\mathbf{x}_1 + (1-t)\mathbf{x}_0 \\ \boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1) = \epsilon \end{cases}$$

image credit: A Visual Dive into Conditional Flow Matching

Two-Sided Conditioning

$$p_0(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_0); \quad p_1(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1)$$

Gaussian Conditional Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(\boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1), \boldsymbol{\sigma}_t^2(\mathbf{x}_0, \mathbf{x}_1)\right); \quad \mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1) + \boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1) \odot \boldsymbol{\epsilon}$$

Let's consider straight conditional paths:

$$\begin{cases} \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1) = t\mathbf{x}_1 + (1-t)\mathbf{x}_0 \\ \boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1) = \epsilon \end{cases} \quad \Rightarrow \quad \begin{cases} p_0(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_0) \\ p_1(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1) \end{cases}$$

image credit: A Visual Dive into Conditional Flow Matching

Two-Sided Conditioning

$$p_0(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_0); \quad p_1(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1)$$

Gaussian Conditional Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(\boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1), \boldsymbol{\sigma}_t^2(\mathbf{x}_0, \mathbf{x}_1)\right); \quad \mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1) + \boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1) \odot \boldsymbol{\epsilon}$$

Let's consider straight conditional paths:

$$\begin{cases} \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1) = t\mathbf{x}_1 + (1-t)\mathbf{x}_0 \\ \boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1) = \epsilon \end{cases} \Rightarrow \begin{cases} p_0(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_0) \\ p_1(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \delta(\mathbf{x} - \mathbf{x}_1) \end{cases}$$

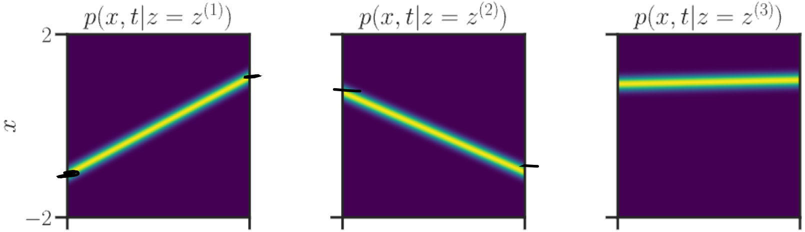


image credit: A Visual Dive into Conditional Flow Matching

Flow Matching: One-Sided vs. Two-Sided Conditioning

$$\mathbf{z} = \mathbf{x}_1 \qquad\qquad \mathbf{z} = (\mathbf{x}_0, \mathbf{x}_1)$$

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1, (1-t)^2\mathbf{I}\right) \quad p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \epsilon^2\mathbf{I}\right)$$

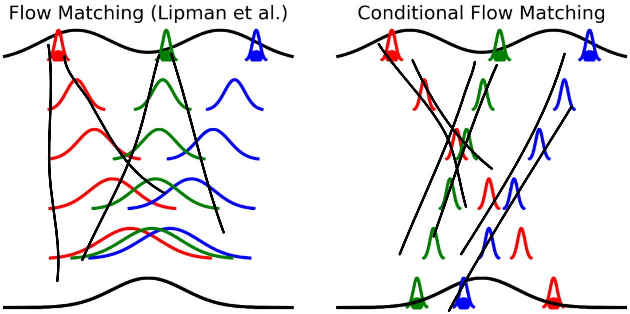
$$\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0 \qquad\qquad \mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$$

Tong A., et al. Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport, 2023

Flow Matching: One-Sided vs. Two-Sided Conditioning

$$\mathbf{z} = \mathbf{x}_1$$
$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1, (1-t)^2\mathbf{I}\right)$$
$$\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$$

$$\mathbf{z} = (\mathbf{x}_0, \mathbf{x}_1)$$
$$p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \epsilon^2\mathbf{I}\right)$$
$$\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$$



Tong A., et al. Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport, 2023

Two-Sided Conditioning: Vector Field and Objective

$$p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \epsilon^2\mathbf{I}\right); \quad \mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$$

Conditional Vector Field

$$\frac{d\mathbf{x}_t}{dt} = \boxed{\mathbf{v}(\mathbf{x}_t, \mathbf{x}_0, \mathbf{x}_1, t)} = \underbrace{\boldsymbol{\mu}'_t(\mathbf{x}_0, \mathbf{x}_1)}_{x_1 - x_0} + \frac{\boldsymbol{\sigma}'_t(\mathbf{x}_0, \mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1))$$

Lipman Y., et al. Flow Matching for Generative Modeling, 2022

Two-Sided Conditioning: Vector Field and Objective

$$p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \epsilon^2 \mathbf{I}\right); \quad \mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$$

Conditional Vector Field

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, \mathbf{x}_0, \mathbf{x}_1, t) = \boldsymbol{\mu}'_t(\mathbf{x}_0, \mathbf{x}_1) + \frac{\boldsymbol{\sigma}'_t(\mathbf{x}_0, \mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1))$$

$$\boxed{\mathbf{v}(\mathbf{x}_t, \mathbf{x}_0, \mathbf{x}_1, t) = \mathbf{x}_1 - \mathbf{x}_0}$$

Lipman Y., et al. Flow Matching for Generative Modeling, 2022

Two-Sided Conditioning: Vector Field and Objective

$$p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \epsilon^2 \mathbf{I}\right); \quad \boxed{\mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0}$$

Conditional Vector Field

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, \mathbf{x}_0, \mathbf{x}_1, t) = \boldsymbol{\mu}'_t(\mathbf{x}_0, \mathbf{x}_1) + \frac{\boldsymbol{\sigma}'_t(\mathbf{x}_0, \mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1))$$

$$\mathbf{v}(\mathbf{x}_t, \mathbf{x}_0, \mathbf{x}_1, t) = \mathbf{x}_1 - \mathbf{x}_0$$

Conditional Flow Matching

$$\boxed{\begin{aligned} &\mathbb{E}_{t \sim U[0,1]} \mathbb{E}_{\mathbf{z} \sim p(\mathbf{z})} \mathbb{E}_{\mathbf{x} \sim p_t(\mathbf{x}|\mathbf{z})} \left\| \mathbf{v}(\mathbf{x}, \mathbf{z}, t) - \mathbf{v}_{\theta}(\mathbf{x}, t) \right\|^2 = \\ &\quad \mathbb{E}_{t \sim U[0,1]} \boxed{\mathbb{E}_{(\mathbf{x}_0, \mathbf{x}_1) \sim p(\mathbf{x}_0, \mathbf{x}_1)}} \mathbb{E}_{\mathbf{x} \sim p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1)} \underbrace{\left\| (\mathbf{x}_1 - \mathbf{x}_0) - \mathbf{v}_{\theta}(\mathbf{x}, t) \right\|^2} \end{aligned}}$$

$p(x_0)p(x_1)$

Lipman Y., et al. Flow Matching for Generative Modeling, 2022

Two-Sided Conditioning: Vector Field and Objective

$$p_t(\mathbf{x}|\mathbf{x}_0, \mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1 + (1-t)\mathbf{x}_0, \epsilon^2\mathbf{I}\right); \quad \mathbf{x}_t = t\mathbf{x}_1 + (1-t)\mathbf{x}_0$$

Conditional Vector Field

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, \mathbf{x}_0, \mathbf{x}_1, t) = \boldsymbol{\mu}'_t(\mathbf{x}_0, \mathbf{x}_1) + \frac{\boldsymbol{\sigma}'_t(\mathbf{x}_0, \mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_0, \mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_0, \mathbf{x}_1))$$

$$\mathbf{v}(\mathbf{x}_t, \mathbf{x}_0, \mathbf{x}_1, t) = \mathbf{x}_1 - \mathbf{x}_0$$

$\boxed{p_0(x)} \longleftrightarrow \boxed{p_1(x)}$

Conditional Flow Matching

$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{z})}\left\|\mathbf{v}(\mathbf{x}, \mathbf{z}, t) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t)\right\|^2 =$$
$$\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{(\mathbf{x}_0,\mathbf{x}_1)\sim p(\mathbf{x}_0,\mathbf{x}_1)}\mathbb{E}_{\mathbf{x}\sim p_t(\mathbf{x}|\mathbf{x}_0,\mathbf{x}_1)}\left\|(\mathbf{x}_1 - \mathbf{x}_0) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t)\right\|^2$$

- This yields the same procedure as for one-sided conditioning!
- Now, we do not require that $\boxed{p_0(\mathbf{x})}$ is necessarily $\boxed{\mathcal{N}(0, \mathbf{I})}$.

Lipman Y., et al. Flow Matching for Generative Modeling, 2022

CFM with Two-Sided Conditioning: Training and Sampling

- This conditioning allows us to transport any distribution $p_0(\mathbf{x})$ to any distribution $p_1(\mathbf{x})$.
- It's possible to apply this approach to paired tasks, e.g., style transfer.

Lipman Y., et al. Flow Matching for Generative Modeling, 2022

CFM with Two-Sided Conditioning: Training and Sampling

- This conditioning allows us to transport any distribution $p_0(\mathbf{x})$ to any distribution $p_1(\mathbf{x})$.
- It's possible to apply this approach to paired tasks, e.g., style transfer.

Training Procedure

1. Sample $(\mathbf{x}_0, \mathbf{x}_1) \sim p(\mathbf{x}_0, \mathbf{x}_1)$.
2. Sample time $t \sim U[0, 1]$.
3. Compute the noisy image $\mathbf{x}_t = t\mathbf{x}_1 + (1 - t)\mathbf{x}_0$.
4. Compute the loss $\mathcal{L} = \|(\mathbf{x}_1 - \mathbf{x}_0) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)\|^2$.

Lipman Y., et al. Flow Matching for Generative Modeling, 2022

CFM with Two-Sided Conditioning: Training and Sampling

- This conditioning allows us to transport any distribution $p_0(\mathbf{x})$ to any distribution $p_1(\mathbf{x})$.
- It's possible to apply this approach to paired tasks, e.g., style transfer.

$= p(x_0)\,p(x_1)$ $\qquad \mathcal{N}(0, I) \leftrightarrow p(x)$

Training Procedure

1. Sample $\boxed{(\mathbf{x}_0, \mathbf{x}_1)} \sim p(\mathbf{x}_0, \mathbf{x}_1)$.
2. Sample time $t \sim U[0, 1]$.
3. Compute the noisy image $\mathbf{x}_t = t\mathbf{x}_1 + (1 - t)\mathbf{x}_0$.
4. Compute the loss $\boxed{\mathcal{L} = \|(\mathbf{x}_1 - \mathbf{x}_0) - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)\|^2}$.

Sampling

1. Sample $\mathbf{x}_0 \sim p_0(\mathbf{x})$. $\qquad x_1 \sim p_1(x)$
2. Solve the ODE to obtain $\mathbf{x}_1$:

$$\mathbf{x}_1 = \psi_1(\mathbf{x}_0) = \texttt{ODESolve}_v(\mathbf{x}_0, \boldsymbol{\theta}, t_0 = 0, t_1 = 1)$$

Lipman Y., et al. Flow Matching for Generative Modeling, 2022

Stable Diffusion 3: Scalable Flow Matching

Esser P., et al. Scaling Rectified Flow Transformers for High-Resolution Image Synthesis, 2024

Outline

Score-Based Generative Models through SDEs

Training

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x}(0))}\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{q(\mathbf{x}(t)|\mathbf{x}(0))}\left\|\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)-\nabla_{\mathbf{x}(t)}\log q(\mathbf{x}(t)|\mathbf{x}(0))\right\|_2^2$$

Lipman Y., et al. Flow Matching for Generative Modeling, 2022

Score-Based Generative Models through SDEs

Training

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x}(0))}\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{q(\mathbf{x}(t)|\mathbf{x}(0))}\left\|\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}(t),t)-\nabla_{\mathbf{x}(t)}\log q(\mathbf{x}(t)|\mathbf{x}(0))\right\|_2^2$$

Variance Exploding SDE (NCSN)

$$q(\mathbf{x}(t)|\mathbf{x}(0))=\mathcal{N}\left(\mathbf{x}(0),\left[\sigma^2(t)-\sigma^2(0)\right]\cdot\mathbf{I}\right),\quad \sigma(0)=0$$

Variance Preserving SDE (DDPM)

$$q(\mathbf{x}(t)|\mathbf{x}(0))=\mathcal{N}\left(\mathbf{x}(0)\alpha(t),\left(1-\alpha(t)^2\right)\cdot\mathbf{I}\right);\quad \alpha(t)=e^{-\frac{1}{2}\int_0^t\beta(s)ds}$$

Diff. $t=0$ p_{data}

$t=1$ $\mathcal{N}(0,I)$

FM $t=0$ $\mathcal{N}(0,I)$

$t=1$ p_{data}

Lipman Y., et al. *Flow Matching for Generative Modeling*, 2022

Score-Based Generative Models through SDEs

Training

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x}(0))}\mathbb{E}_{t\sim U[0,1]}\mathbb{E}_{q(\mathbf{x}(t)|\mathbf{x}(0))}\left\|\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}(t),t)-\nabla_{\mathbf{x}(t)}\log q(\mathbf{x}(t)|\mathbf{x}(0))\right\|_2^2$$

Variance Exploding SDE (NCSN)

$$q(\mathbf{x}(t)|\mathbf{x}(0)) = \mathcal{N}\left(\mathbf{x}(0), \left[\sigma^2(t) - \sigma^2(0)\right]\cdot \mathbf{I}\right), \quad \sigma(0) = 0$$

Variance Preserving SDE (DDPM)

$$q(\mathbf{x}(t)|\mathbf{x}(0)) = \mathcal{N}\left(\mathbf{x}(0)\alpha(t), \left(1-\alpha(t)^2\right)\cdot \mathbf{I}\right); \quad \alpha(t) = e^{-\frac{1}{2}\int_0^t \beta(s)ds}$$

Pdata

Flow matching uses reverse time direction:

$$\boxed{p_t(\mathbf{x}|\mathbf{x}_1)} = q_{1-t}(\mathbf{x}|\mathbf{x}_0 = \mathbf{x}_1)$$

$X_1 = P_{data}$

Lipman Y., et al. Flow Matching for Generative Modeling, 2022

Score-Based Generative Models through SDEs

$$p_t(\mathbf{x}|\mathbf{x}_1) = q_{1-t}(\mathbf{x}|\mathbf{x}_0 = \mathbf{x}_1)$$

$\sigma(0)=0$

$$\textbf{VE (NCSN):}\ p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\mathbf{x}_1, \sigma_{1-t}^2 \cdot \mathbf{I}\right)$$

$$\textbf{VP (DDPM):}\ p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\alpha_{1-t}\mathbf{x}_1, \left(1 - \alpha_{1-t}^2\right) \cdot \mathbf{I}\right)$$

$\alpha(1-t) = e^{-\int \cdots}$

Score-Based Generative Models through SDEs

$$p_t(\mathbf{x}|\mathbf{x}_1) = q_{1-t}(\mathbf{x}|\mathbf{x}_0 = \mathbf{x}_1)$$

VE (NCSN): $p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\mathbf{x}_1, \sigma_{1-t}^2 \cdot \mathbf{I}\right)$

VP (DDPM): $p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\alpha_{1-t}\mathbf{x}_1, \left(1 - \alpha_{1-t}^2\right) \cdot \mathbf{I}\right)$

Flow Matching Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1, (1-t)^2\mathbf{I}\right); \quad \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \frac{\mathbf{x}_1 - \mathbf{x}_t}{1-t}$$

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \boldsymbol{\mu}'_t(\mathbf{x}_1) + \frac{\boldsymbol{\sigma}'_t(\mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_1))$$

$v(x_t, x, t)$

Score-Based Generative Models through SDEs

$$p_t(\mathbf{x}|\mathbf{x}_1) = q_{1-t}(\mathbf{x}|\mathbf{x}_0 = \mathbf{x}_1)$$

$$\textbf{VE (NCSN):}\ p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\mathbf{x}_1, \sigma_{1-t}^2 \cdot \mathbf{I}\right)$$

$$\textbf{VP (DDPM):}\ p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\alpha_{1-t}\mathbf{x}_1, \left(1 - \alpha_{1-t}^2\right) \cdot \mathbf{I}\right)$$

Flow Matching Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(t\mathbf{x}_1, (1-t)^2\mathbf{I}\right); \quad \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \frac{\mathbf{x}_1 - \mathbf{x}_t}{1-t}$$

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \boldsymbol{\mu}'_t(\mathbf{x}_1) + \frac{\boldsymbol{\sigma}'_t(\mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_1))$$

Let's derive the conditional vector fields for VE (NCSN) and VP (DDPM).

Flow Matching vs Score-Based SDE Models

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \boldsymbol{\mu}'_t(\mathbf{x}_1) + \frac{\boldsymbol{\sigma}'_t(\mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_1))$$

Lipman Y., et al. Flow Matching for Generative Modeling, 2022

Flow Matching vs Score-Based SDE Models

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \boldsymbol{\mu}'_t(\mathbf{x}_1) + \frac{\boldsymbol{\sigma}'_t(\mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_1))$$

Variance Exploding SDE Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\mathbf{x}_1, \sigma^2_{1-t}\mathbf{I}\right) \quad \Rightarrow \quad \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = -\frac{\sigma'_{1-t}}{\sigma_{1-t}}(\mathbf{x}_t - \mathbf{x}_1)$$

$\mu' = 0$

Lipman Y., et al. *Flow Matching for Generative Modeling*, 2022

Flow Matching vs Score-Based SDE Models

$$\frac{d\mathbf{x}_t}{dt} = \mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \boldsymbol{\mu}'_t(\mathbf{x}_1) + \frac{\boldsymbol{\sigma}'_t(\mathbf{x}_1)}{\boldsymbol{\sigma}_t(\mathbf{x}_1)} \odot (\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_1))$$

$v = \boxed{x_1 - x_0}$ $\boxed{\frac{x_1 - x}{1 -}}$

Variance Exploding SDE Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\mathbf{x}_1, \sigma_{1-t}^2 \mathbf{I}\right) \quad \Rightarrow \quad \boxed{\mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = -\boxed{\frac{\sigma'_{1-t}}{\sigma_{1-t}}}(\mathbf{x}_t - \mathbf{x}_1)}$$

F

Variance Preserving SDE Probability Path

$$p_t(\mathbf{x}|\mathbf{x}_1) = \mathcal{N}\left(\alpha_{1-t}\mathbf{x}_1, (1 - \alpha_{1-t}^2)\mathbf{I}\right) \Rightarrow \boxed{\mathbf{v}(\mathbf{x}_t, \mathbf{x}_1, t) = \boxed{\frac{\alpha'_{1-t}}{1 - \alpha_{1-t}^2}}(\alpha_{1-t}\mathbf{x}_t - \mathbf{x}_1)}$$

Thus, VE/VP SDE models correspond to particular choices of the Gaussian probability path within the flow matching framework.

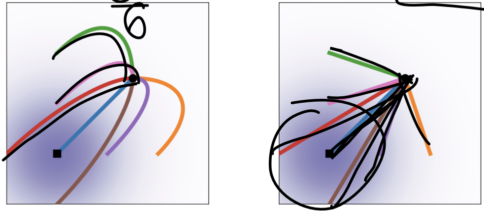
Lipman Y., et al. Flow Matching for Generative Modeling, 2022

Flow Matching vs Score-Based SDE Models

$\boxed{\text{ODESolve}}$

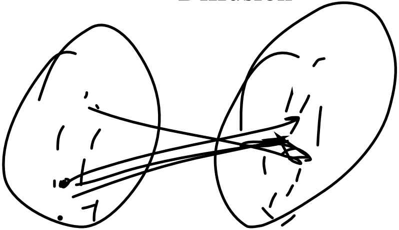
Trajectories

$\frac{\sigma'}{\sigma}$



Diffusion OT

training



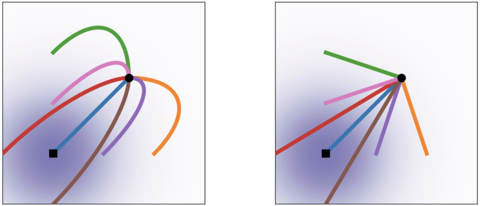
$x|z$

v_θ

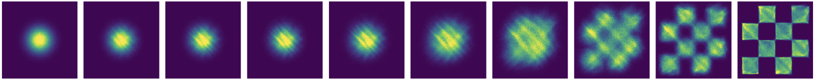
x

Flow Matching vs Score-Based SDE Models

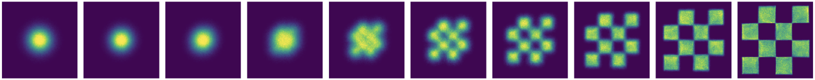
Trajectories



Diffusion   OT



Score matching w/ Diffusion



Flow Matching w/ OT

Lipman Y., et al. Flow Matching for Generative Modeling, 2022

Outline

Discrete or Continuous Diffusion Models?

Reminder: Diffusion models define a forward corruption process and a reverse denoising process. Previously, we studied diffusion models with continuous states $\mathbf{x}(t) \in \mathbb{R}^m$.

Continuous state space

- **Discrete time** $t \in \{0, 1, \ldots, T\}$ $\Rightarrow$ **DDPM / NCSN**.
- **Continuous time** $t \in [0, 1]$ $\Rightarrow$ **Score-based SDE models.**

SDE

Discrete or Continuous Diffusion Models?

Reminder: Diffusion models define a forward corruption process and a reverse denoising process. Previously, we studied diffusion models with continuous states $\mathbf{x}(t) \in \mathbb{R}^m$.

Continuous state space

- **Discrete time** $t \in \{0, 1, \dots, T\}$ $\Rightarrow$ **DDPM / NCSN**.
- **Continuous time** $t \in [0, 1]$ $\Rightarrow$ **Score-based SDE models**.

Now we turn to diffusion over discrete-value states $\boxed{\mathbf{x}(t)} \in \{1, \dots, K\}^m$.

Discrete state space

- $\boxed{\textbf{Discrete time } t \in \{0, 1, \dots, T\}}$.
- **Continuous time** $t \in [0, 1]$.

Let's discuss why we need discrete diffusion models.

Why Discrete Diffusion Models?

While autoregressive (AR) models dominate discrete-data domains (e.g., text or sequences), they have fundamental limitations.

https://aaronlou.com/blog/2024/discrete-diffusion/

Why Discrete Diffusion Models?

While autoregressive (AR) models dominate discrete-data domains (e.g., text or sequences), they have fundamental limitations.

Key advantages of discrete diffusion

- **Parallel generation:** diffusion enables sampling all tokens simultaneously, unlike AR's strictly left-to-right process.

$$x_t \mid x_{1:t-1}$$

https://aaronlou.com/blog/2024/discrete-diffusion/

Why Discrete Diffusion Models?

While autoregressive (AR) models dominate discrete-data domains (e.g., text or sequences), they have fundamental limitations.

Key advantages of discrete diffusion

- **Parallel generation:** diffusion enables sampling all tokens simultaneously, unlike AR's strictly left-to-right process.
- **Flexible infilling:** diffusion can mask arbitrary parts of a sequence and reconstruct them, rather than generating only from prefix to suffix.

---

https://aaronlou.com/blog/2024/discrete-diffusion/

Why Discrete Diffusion Models?

While autoregressive (AR) models dominate discrete-data domains (e.g., text or sequences), they have fundamental limitations.

Key advantages of discrete diffusion

- **Parallel generation:** diffusion enables sampling all tokens simultaneously, unlike AR's strictly left-to-right process.
- **Flexible infilling:**. diffusion can mask arbitrary parts of a sequence and reconstruct them, rather than generating only from prefix to suffix.
- **Robustness:** diffusion avoids the $\boxed{\text{"exposure bias"}}$ caused by teacher forcing in AR training.

$x_t (y_{1:t-1}$

https://aaronlou.com/blog/2024/discrete-diffusion/

Why Discrete Diffusion Models?

While autoregressive (AR) models dominate discrete-data domains (e.g., text or sequences), they have fundamental limitations.

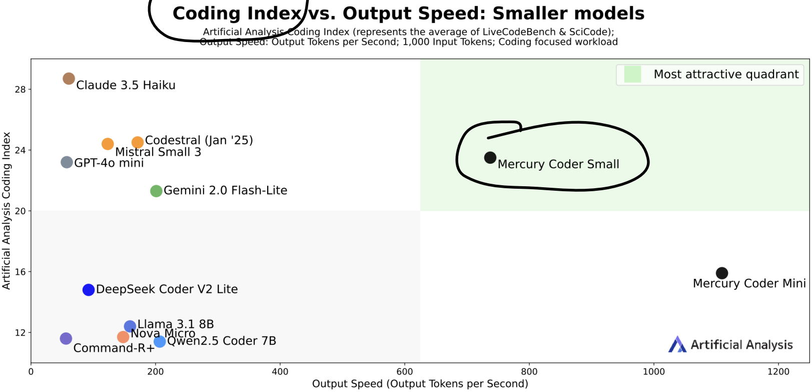
Key advantages of discrete diffusion

- **Parallel generation:** diffusion enables sampling all tokens simultaneously, unlike AR's strictly left-to-right process.
- **Flexible infilling:**. diffusion can mask arbitrary parts of a sequence and reconstruct them, rather than generating only from prefix to suffix.
- **Robustness:** diffusion avoids the "exposure bias" caused by teacher forcing in AR training.
- **Unified framework:** diffusion generalizes naturally to discrete domains that do not suit continuous Gaussian noise.

---

https://aaronlou.com/blog/2024/discrete-diffusion/

2025 – Big Bang of Discrete Diffusion Models



Mercury?

Khanna S. et al. *Mercury: Ultra-fast language models based on diffusion*, 2025.

Outline

1. Two-Sided Conditioning

2. Link between Flow Matching and Score-Based Models

3. Discrete Diffusion Models

4. Forward Discrete Process

Forward Discrete Process

Continuous Diffusion Markov Chain

In continuous diffusion, the forward Markov chain is defined by progressively corrupting data with Gaussian noise:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\underbrace{\sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}}).$$

$t \to \infty$

$\mathcal{N}(0, I)$

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Forward Discrete Process

Continuous Diffusion Markov Chain

In continuous diffusion, the forward Markov chain is defined by progressively corrupting data with Gaussian noise:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}).$$

Discrete Diffusion Markov Chain

For discrete data, we instead define a Markov chain over categorical states:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \boxed{\mathrm{Cat}(\mathbf{Q}_t\underline{\mathbf{x}_{t-1}}),}$$

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Forward Discrete Process

Continuous Diffusion Markov Chain

In continuous diffusion, the forward Markov chain is defined by progressively corrupting data with Gaussian noise:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}).$$

$\mathcal{N}(0, I)$

Discrete Diffusion Markov Chain

For discrete data, we instead define a Markov chain over categorical states:

$\mathcal{U}\{1, \dots, K\}$

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathrm{Cat}(\mathbf{Q}_t\mathbf{x}_{t-1}),$$

- Each $\mathbf{x}_t \in \{0, 1\}^K$ is a **one-hot vector** encoding the categorical state (it is just one token).
- What is the transition matrix $\mathbf{Q}_t$?

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Forward Process over Time

Transition Matrix

$\mathbf{Q}_t \in [0,1]^{K\times K}$ is a **transition matrix** where each column gives transition probabilities from one state to all others, and columns sum to 1:

$$[\mathbf{Q}_t]_{ij} = q(x_t = i | x_{t-1} = j), \qquad \sum_{i=1}^{K} [\mathbf{Q}_t]_{ij} = 1.$$

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Forward Process over Time

Transition Matrix

$\mathbf{Q}_t \in [0,1]^{K\times K}$ is a **transition matrix** where each column gives transition probabilities from one state to all others, and columns sum to 1:

$$[\mathbf{Q}_t]_{ij} = q(x_t = i | x_{t-1} = j), \qquad \sum_{i=1}^{K} [\mathbf{Q}_t]_{ij} = 1.$$

- The forward diffusion gradually destroys information through repeated random transitions.

$Q_t x_{t-1}$

Austin J. et al. *Structured denoising diffusion models in discrete state-spaces, 2021.*

Forward Process over Time

Transition Matrix

$\mathbf{Q}_t \in [0,1]^{K\times K}$ is a **transition matrix** where each column gives transition probabilities from one state to all others, and columns sum to 1:

$$[\mathbf{Q}_t]_{ij} = q(x_t = i | x_{t-1} = j), \qquad \sum_{i=1}^{K} [\mathbf{Q}_t]_{ij} = 1.$$

- The forward diffusion gradually destroys information through repeated random transitions.
- Applying the transition t times yields the marginal distribution:

$$\boxed{q(\mathbf{x}_t|\mathbf{x}_0)} = \mathrm{Cat}(\underbrace{\mathbf{Q}_{1:t}}_{}\mathbf{x}_0), \qquad \mathbf{Q}_{1:t} = \boxed{\mathbf{Q}_t\mathbf{Q}_{t-1}\cdots\mathbf{Q}_1.}$$

Q^t

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Forward Process over Time

Transition Matrix

$\mathbf{Q}_t \in [0,1]^{K \times K}$ is a **transition matrix** where each column gives transition probabilities from one state to all others, and columns sum to 1:

$$[\mathbf{Q}_t]_{ij} = q(x_t = i | x_{t-1} = j), \qquad \sum_{i=1}^{K} [\mathbf{Q}_t]_{ij} = 1.$$

- The forward diffusion gradually destroys information through repeated random transitions.
- Applying the transition t times yields the marginal distribution:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{x}_0), \qquad \mathbf{Q}_{1:t} = \mathbf{Q}_t \mathbf{Q}_{t-1} \cdots \mathbf{Q}_1.$$

- As $t \to T$, the process drives the data toward a stationary distribution.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Forward Process over Time

Transition Matrix

$\mathbf{Q}_t \in [0,1]^{K\times K}$ is a **transition matrix** where each column gives transition probabilities from one state to all others, and columns sum to 1:

$$[\mathbf{Q}_t]_{ij} = q(x_t = i | x_{t-1} = j), \qquad \sum_{i=1}^{K} [\mathbf{Q}_t]_{ij} = 1.$$

- The forward diffusion gradually destroys information through repeated random transitions.
- Applying the transition t times yields the marginal distribution:

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{x}_0), \qquad \mathbf{Q}_{1:t} = \mathbf{Q}_t\mathbf{Q}_{t-1}\cdots\mathbf{Q}_1.$$

- As $t \to T$, the process drives the data toward a stationary distribution.
- We design the transition matrices $\mathbf{Q}_t$ to achieve this behavior.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Transition Matrix

- The choice of $\mathbf{Q}_t$ determines how information is erased and what the stationary distribution becomes.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

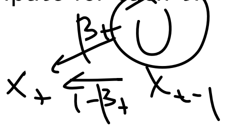
Transition Matrix

- The choice of $\mathbf{Q}_t$ determines how information is erased and what the stationary distribution becomes.
- $\mathbf{Q}_t$ and $\mathbf{Q}_{1:t}$ should be easy to compute for each t.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Transition Matrix

- The choice of $\mathbf{Q}_t$ determines how information is erased and what the stationary distribution becomes.
- $\mathbf{Q}_t$ and $\mathbf{Q}_{1:t}$ should be easy to compute for each t.



Common choices

- **Uniform diffusion**

$$\mathbf{Q}_t = (1-\beta_t)\mathbf{I} + \beta_t \mathbf{U}, \qquad \mathbf{U}_{ij} = \frac{1}{K}.$$

Each token is replaced by a uniformly random symbol with probability β_t. The stationary distribution is uniform noise.

$$Q_\infty$$

$$Q_t = \begin{pmatrix} 1-\beta_t\frac{K-1}{K} \\ \frac{\beta_t}{K} \\ \cdots \\ \cdots \\ \cdots \end{pmatrix}$$

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Transition Matrix

- The choice of $\mathbf{Q}_t$ determines how information is erased and what the stationary distribution becomes.
- $\mathbf{Q}_t$ and $\mathbf{Q}_{1:t}$ should be easy to compute for each t.

$$Q_t = \begin{array}{c} \\ \\ m \\ \\ \end{array}\begin{pmatrix} 1-\beta_t & & 0 \\ 0 & 1-\beta_t & 0 \\ \beta_t & \cdots\ \beta_t & 1 \\ & 0 & 0 \end{pmatrix}$$

Common choices

- **Uniform diffusion**

$$\mathbf{Q}_t = (1-\beta_t)\mathbf{I} + \beta_t \mathbf{U}, \qquad \mathbf{U}_{ij} = \frac{1}{K}.$$

Each token is replaced by a uniformly random symbol with probability β_t. The stationary distribution is uniform noise.

- **Absorbing diffusion**

$$\mathbf{Q}_t = \boxed{(1-\beta_t)\mathbf{I}} + \beta_t \underbrace{\mathbf{e}_m}_{} \mathbf{1}^\top.$$

$$x_t \xleftarrow{(1-\beta_t)} x_{t-1}$$
$$x_t \xleftarrow{\beta_t} x_m$$

Tokens are gradually replaced by a special mask m; the stationary distribution is fully masked.

Austin J. et al. Structured denoising diffusion models in discrete state-spaces, 2021.

Transition Matrix

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{x}_0), \qquad \mathbf{Q}_{1:t} = \mathbf{Q}_t\mathbf{Q}_{t-1}\cdots\mathbf{Q}_1.$$

Transition Matrix

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{x}_0), \qquad \mathbf{Q}_{1:t} = \mathbf{Q}_t\mathbf{Q}_{t-1}\cdots\mathbf{Q}_1.$$

Uniform Diffusion

$$\mathbf{Q}_t = (1-\beta_t)\mathbf{I} + \beta_t\mathbf{U}, \qquad \mathbf{U}_{ij} = \frac{1}{K}.$$

$$\boxed{\mathbf{Q}_{1:t} = \bar{\alpha}_t\mathbf{I} + (1-\bar{\alpha}_t)\mathbf{U},} \quad \boxed{\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s).}$$

Transition Matrix

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathrm{Cat}(\mathbf{Q}_{1:t}\mathbf{x}_0), \qquad \mathbf{Q}_{1:t} = \mathbf{Q}_t\mathbf{Q}_{t-1}\cdots\mathbf{Q}_1.$$

Uniform Diffusion

$$\mathbf{Q}_t = (1-\beta_t)\mathbf{I} + \beta_t\mathbf{U}, \qquad \mathbf{U}_{ij} = \frac{1}{K}.$$

$$\mathbf{Q}_{1:t} = \bar{\alpha}_t\mathbf{I} + (1-\bar{\alpha}_t)\mathbf{U}, \quad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s).$$

- Each token retains its original value with prob. $\bar{\alpha}_t$.
- It becomes uniformly random with prob. $(1-\bar{\alpha}_t)$.
- As $t \to T$, the process converges to the stationary uniform distribution.

Transition Matrix

Absorbing Diffusion

$$\mathbf{Q}_t = (1-\beta_t)\mathbf{I} + \beta_t\,\mathbf{e}_m\mathbf{1}^\top,$$

$$\boxed{\mathbf{Q}_{1:t} = \underbrace{\bar{\alpha}_t}_{0}\,\mathbf{I} + \underbrace{(1-\bar{\alpha}_t)}_{1}\,\mathbf{e}_m\mathbf{1}^\top,} \qquad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s).$$

$$Q = \begin{array}{c} \\ m \\ \\ \end{array}\begin{pmatrix} & & 0 & & \\ 1 & 1 & 1 & 1 & 1 \\ & & 0 & & \end{pmatrix}$$

Transition Matrix

[MASK]

$\left(\bar{x}_t \rightleftarrows x_m \right)$

Absorbing Diffusion

$$\mathbf{Q}_t = (1-\beta_t)\mathbf{I} + \beta_t\, \mathbf{e}_m \mathbf{1}^\top,$$

$$\mathbf{Q}_{1:t} = \bar{\alpha}_t\, \mathbf{I} + (1-\bar{\alpha}_t)\, \mathbf{e}_m \mathbf{1}^\top, \qquad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s).$$

- Each token retains its original value with prob. $\bar{\alpha}_t$.
- It becomes $\mathbf{e}_m$ with prob. $(1-\bar{\alpha}_t)$.
- As $t \to T$, all tokens converge to the mask state: $q(\mathbf{x}_T) \approx \mathrm{Cat}(\mathbf{e}_m)$.
- This makes the process analogous to **masked language modeling**.

MLM

Austin J. et al., Structured denoising diffusion models in discrete state-spaces, 2021.

Uniform vs. Absorbing Transition Matrix

Aspect	Uniform Diffusion	Absorbing Diffusion
$\mathbf{Q}_t$	$(1-\beta_t)\mathbf{I} + \beta_t\mathbf{U}$	$(1-\beta_t)\mathbf{I} + \beta_t\mathbf{e}_m\mathbf{1}^\top$
$\mathbf{Q}_{1:t}$	$\bar{\alpha}_t\mathbf{I} + (1-\bar{\alpha}_t)\mathbf{U}$	$\bar{\alpha}_t\mathbf{I} + (1-\bar{\alpha}_t)\mathbf{e}_m\mathbf{1}^\top$
$\mathbf{Q}_{1:\infty}$	$\mathbf{U}$	$\mathrm{Cat}(\mathbf{e}_m)$
Interpretation	Random replacement	Gradual masking of tokens
Application	Image diffusion	Text diffusion ≈ Masked LM

Austin J. et al., Structured denoising diffusion models in discrete state-spaces, 2021.

Uniform vs. Absorbing Transition Matrix

Aspect	Uniform Diffusion	Absorbing Diffusion
$\mathbf{Q}_t$	$(1-\beta_t)\mathbf{I} + \beta_t \mathbf{U}$	$(1-\beta_t)\mathbf{I} + \beta_t \mathbf{e}_m \mathbf{1}^\top$
$\mathbf{Q}_{1:t}$	$\bar{\alpha}_t \mathbf{I} + (1-\bar{\alpha}_t)\mathbf{U}$	$\bar{\alpha}_t \mathbf{I} + (1-\bar{\alpha}_t)\mathbf{e}_m \mathbf{1}^\top$
$\mathbf{Q}_{1:\infty}$	$\mathbf{U}$	$\mathrm{Cat}(\mathbf{e}_m)$
Interpretation	Random replacement	Gradual masking of tokens
Application	Image diffusion	Text diffusion $\approx$ Masked LM

Observation

Both schemes gradually destroy information, but differ in their stationary limit. Absorbing diffusion bridges diffusion and masked-language-model objectives.

Austin J. et al., Structured denoising diffusion models in discrete state-spaces, 2021.

Summary

- Two-sided conditioning uses pair conditioning $\mathbf{z} = (\mathbf{x}_0, \mathbf{x}_1)$ and yields the same procedure, but is more general (suitable for paired tasks).
- Diffusion and score-based models are special cases of the flow matching approach, but use curved trajectories.
- Diffusion approach has several key advantages over autoregressive approach.
- Forward discrete diffusion process defines Markov chain with discrete states.
- There are several ways to make it tractable (uniform / absorbing transitions).